

CS396 Project Phase 1 Report

epaData: A Data Management System for EPA Power-Sector Emissions Data

Raaj Banga

Arnav Bawankule

October 2026

Abstract

epaData is a web application that retrieves annual operating and emissions data for U.S. power-plant units from the EPA Clean Air Markets Program Data (CAMPD) API, imports CSV and Excel files after validating them, stores the result in a relational SQLite database, and lets users search, rank, compare, inspect, and download it. The committed database holds 1,619 facilities, 5,204 generating units, and 50,947 unit-year records for the reporting years 2015–2026. Every record can be traced to the retrieval or file that last wrote it. Every import, whether from the API or a file, is first compared with the stored data and is saved only after the user approves it. Rows that fail validation are kept in a data-quality report rather than discarded. Searches can be built from filters, from numeric ranges, or from a plain-English description that a rule-based parser translates into the same filters. This report describes the problem, the functions delivered, the architecture and database design, the technologies used, the most distinctive features, the testing performed, and the limitations that remain for Phase 2.

Contents

1	Project description	4
1.1	The problem	4
1.2	Intended users	4
1.3	Objectives	4
1.4	Scope	5
2	Functions implemented	5
2.1	Home page	6
2.2	Data retrieval from the EPA API	6
2.3	File upload and validation	7
2.4	Search	7
2.5	Facility and unit details	8
2.6	Comparison, map, and audits	8
2.7	Download	8
2.8	Requirement traceability	9
2.9	Planned functions not completed or changed	9

3	System design	10
3.1	Architecture	10
3.2	Life cycle of a search	11
3.3	Database schema	11
3.4	Data flow and provenance	13
4	Technical details	14
4.1	Languages	14
4.2	Database system	14
4.3	Frameworks and libraries	15
4.4	External services	15
4.5	Scripts	15
4.6	Configuration	16
5	Highlighted features	16
5.1	Comparing the database with the EPA API before saving	16
5.2	Upload validation and the data-quality report	18
5.3	Description-based search	19
5.4	Multi-constraint search and ranking in SQL	22
5.5	Physical-sanity audits	24
5.6	Provenance and export	25
6	Testing and results	25
6.1	Approach	25
6.2	Unit tests	26
6.3	Representative test cases	26
6.4	Performance	27
6.5	Known issues	28
6.6	Does the project meet its objectives?	28
7	Discussion	29
7.1	Which techniques and tools worked best for which tasks	29
7.2	Important technical choices and alternatives	29
7.3	Challenges and lessons learned	30
8	Limitations and future improvements	31
8.1	Current limitations	31
8.2	Future improvements	32
9	Individual contributions	33

10 References and acknowledgments	34
10.1 Data	34
10.2 Reused code and assets	34
10.3 Generative AI disclosure	34

1 Project description

1.1 The problem

Fossil-fuel power plants in the United States that fall under federal programs such as the Acid Rain Program and the Cross-State Air Pollution Rule must monitor and report how much fuel they burn, how much electricity they generate, and how much sulfur dioxide (SO₂), nitrogen oxides (NO_x), and carbon dioxide (CO₂) leave their stacks. The U.S. Environmental Protection Agency publishes these reports through the Clean Air Markets Program Data (CAMPD) service, both as downloadable files and through a REST API, the CAM API. The data is detailed and authoritative: for every generating unit it gives annual operating time, gross load, heat input, and emitted mass, together with the unit's fuel, type, and pollution-control equipment.

Getting from this source to an answer is harder than it should be. A question such as “*Which coal units in Kentucky emitted more than 500,000 tons of CO₂ in 2025?*” needs a request against the right endpoint with the right parameters, and joining the result with facility metadata, then filtering on a numeric threshold. A comparison across years needs one request per year. Analysts who receive CAMPD exports as spreadsheets also have no simple way to check those files for missing values, out-of-range numbers, or repeated facility-unit-year records before using them, or to see how a file differs from data they already hold. And once data has been combined from several sources, it is easy to lose track of where each number came from.

1.2 Intended users

The system is designed for three groups:

- **Students and researchers** who need emissions data for coursework or analysis and want to query it without writing API code.
- **Environmental and energy analysts** who combine CAMPD data with their own spreadsheet extracts and need those files validated and reconciled with what they already have.
- **Reviewers and regulators** who must be able to trace a figure back to its source, see when it was retrieved, and see which records were rejected and why.

None of these users is assumed to know SQL or the structure of the CAMPD API. The interface therefore exposes searches as forms, tables, and plain-English descriptions.

1.3 Objectives

The course project description defines twelve learning objectives for the full two-phase project. Phase 1 addresses those concerned with data management:

- O1. Design a web-based information system with a front end and a database back end.
- O2. Develop server routes, page templates, forms, and interactive web components.
- O3. Design and implement a relational database using SQLite and an object-relational mapper.
- O4. Retrieve environmental data from public sources or import downloaded files.
- O5. Validate, clean, transform, and store uploaded data.
- O6. Implement database search, filtering, sorting, and pagination.
- O7. Export selected data and calculated results.

O11. Test and deploy a complete data-driven web application.

O12. Document data sources, assumptions, equations, units, and system limitations.

Objectives O8–O10 (environmental indicators, multi-indicator scoring, and interactive charts of evaluation results) belong to Phase 2. Section 6 returns to this list and states, for each objective, whether it was met.

1.4 Scope

Phase 1 is a data-management system, not an evaluation tool. Within scope are retrieval of annual apportioned emissions from the CAM API, upload of CSV and Excel files that follow the CAMPD column layout, validation and storage in SQLite, search (basic, multi-constraint, ranking, and description-based), display of facility and unit details, comparison of plants and units, and CSV export of datasets, search results, selections, valid and invalid records, and provenance.

Out of scope are the EPA TRACI characterisation factors [1], environmental-impact indicators, and the weighted scoring model, all planned for Phase 2. Sub-annual data (hourly to monthly) is displayed on demand from the API but not stored. The application runs locally; a hosted deployment was considered and dropped (Section 8).

2 Functions implemented

epaData is a single web page. Instead of separate pages, the specification’s pages map to a home view, four explorer tabs (Facilities, Units, Map, Audits), and dialogs opened from the header: Retrieve, Upload, and Download, plus the facility, unit, and comparison dialogs opened from the tables. Figure 1 shows the home view. This section describes each function from the user’s point of view; Section 3 explains how they are built.

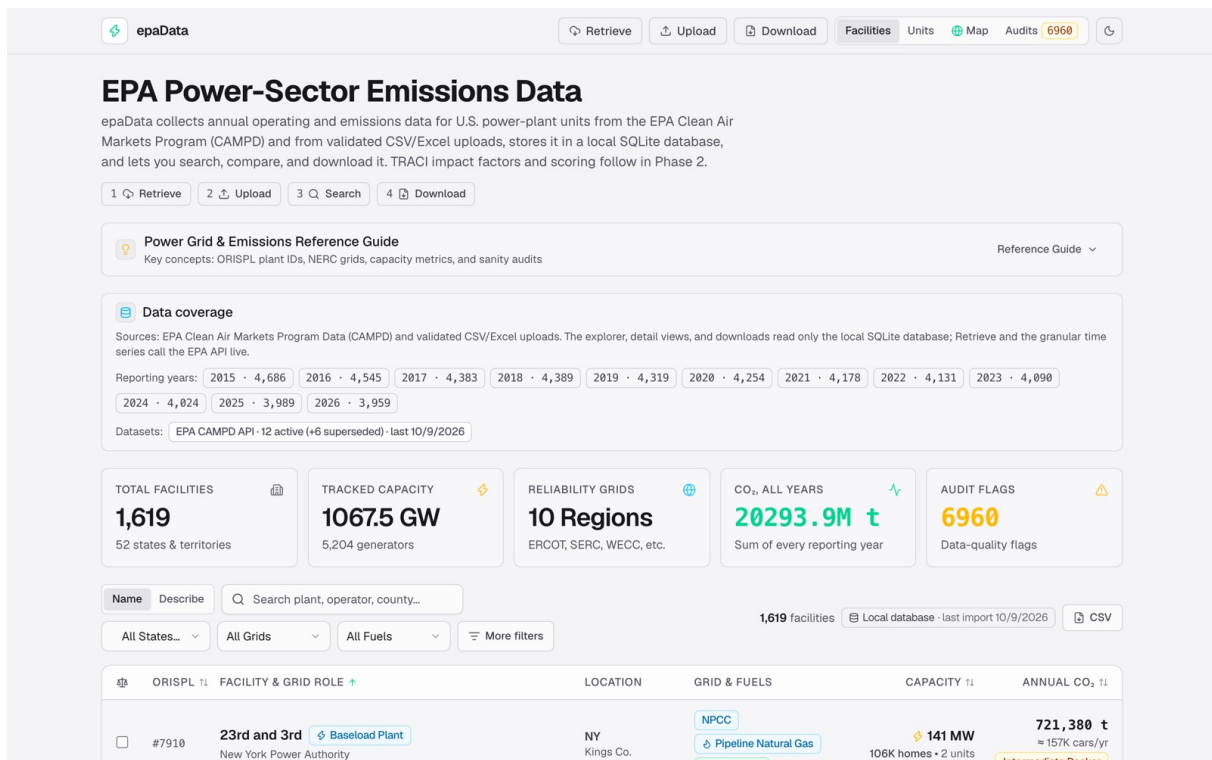


Figure 1: Home view: introduction, numbered links to the four main functions, data coverage per reporting year and source, summary tiles, and the start of the Facilities explorer.

2.1 Home page

The home view opens with a short introduction to the project and its data sources, followed by four numbered buttons in workflow order: Retrieve, Upload, Search, and Download. A coverage strip lists the number of unit-year records for each reporting year and summarises the datasets by source, for example “EPA CAMPD API · 12 active (+6 superseded)”. It also states which parts of the application read the local database and which call the EPA API live. Five tiles summarise the database: facilities, installed capacity, reliability regions, total CO₂ over all years, and the number of data-quality flags.

2.2 Data retrieval from the EPA API

The **Retrieve** dialog names its method (EPA CAM API, apportioned annual emissions) and offers the filters required by the specification: a range of reporting years, state, facility IDs, fuel type, unit type, and control technology. The fuel, unit-type, and control lists are loaded from CAMPD’s master-data endpoints, so every choice is a value the API accepts. While the user edits the form, a panel titled *Already in the database* shows how many unit-years the local database holds for those years and filters and which dataset last wrote them.

Pressing **Preview** fetches the data without saving anything. The dialog then reports, per year, how many rows were received and how many are new, changed, unchanged, or dropped, and lists example rows in each category. For changed rows it shows the stored and incoming value of every field that differs. **Approve & save** then stores the year as a new dataset; **Cancel** discards the preview. A history table lists every past retrieval with its parameters, date, status, and counts. The API key is read from a server-side environment variable and never reaches the browser. Section 5.1 walks through an example.

2.3 File upload and validation

The **Upload** dialog accepts a CSV or Excel file up to 100 MB, either dragged onto the dialog or selected with a file picker. The extension and size are checked in the browser and again on the server. A Python script then reads the file, matches its headers to the database columns, validates every row, and detects repeated facility-unit-year records. The dialog shows the result before anything is written:

- counts of rows, valid rows, rejected rows, duplicates, and physical-sanity flags;
- the tables the import will write and how its records compare with the database (new, changed, unchanged);
- a row preview with a status per row, the rejected rows with the column, value, and reason, the duplicates and how each is resolved, and the column mapping.

The user approves or cancels. An approved import stores the valid records and the rejected and duplicate rows. It also stores the upload's metadata and a copy of the original file. The rejected rows can be downloaded straight from the confirmation. Section 5.2 shows this on the committed sample file.

2.4 Search

The Facilities and Units tabs share one filter bar. The Facilities tab shows one row per plant with totals; the Units tab shows one row per unit and reporting year, the grain of the underlying records.

Basic search. A text box searches facility names, owners and operators, and counties, and also matches a facility ID. Selects in the toolbar filter by state, NERC region, and primary fuel; *More filters* adds facility ID, unit ID, reporting year, county, secondary fuel, unit type, SO₂/NO_x/PM control, operating status, audit flag and severity, and the origin of the record (API or file upload).

Multi-constraint and range search. All filters combine with AND. Under *More filters*, inclusive minimum and maximum bounds are available for reporting year, operating time, gross load, heat input, and CO₂, SO₂, and NO_x mass. Every active filter is also shown as a removable chip above the table.

Description-based search. A *Name/Describe* switch turns the search box into a sentence input. A description such as “Find coal units in Kentucky with high CO₂ emissions” is translated into the same filters, which appear as chips and in the filter controls so that the user can refine them. Words that could not be interpreted are listed rather than ignored. Section 5.3 describes the method.

Ranking and comparison. Any metric column can be sorted. A *Ranking* control keeps the first N rows of that order, overall or per state, which covers the specification's top- N , bottom- N , and group-and-rank searches. Facilities and units can be ticked and compared side by side (2–4 at a time). Historical searches use the year range or the unit dialog, which lists every reporting year of a unit.

Result handling. Results are paginated (10 to 100 rows per page), and every column with a meaningful order is sortable. Each unit row opens the unit dialog. A **CSV** button downloads every matching row, across all pages. The tab, filters, sort, and page are kept in the URL query string, so a search can be bookmarked, shared, and reloaded.

2.5 Facility and unit details

Clicking a facility opens a dialog with its identification (ID, county, state, EPA and NERC region, owner), its units with their fuels and controls, its year-by-year operation and emissions, and its audit flags. A granular view in the same dialog shows hourly, daily, weekly, or monthly data fetched live from the API, and yearly totals from the database. The unit dialog (Figure 12) shows identification, fuel and controls, programs, a table of every reporting year with the dataset each record came from, and any sanity flags.

2.6 Comparison, map, and audits

The comparison dialog (Figure 2) places two to four facilities or units side by side for the latest reporting year: capacity, generation, operating hours, capacity factor, carbon intensity, heat rate, fuels, and control coverage. The Map tab draws every facility with coordinates either on a three-dimensional globe or on a two-dimensional Leaflet map, sized by capacity or CO₂ and coloured by fuel (Figure 3). The Audits tab lists every physical-sanity flag with its rule, severity, and explanation, paginated and filterable.

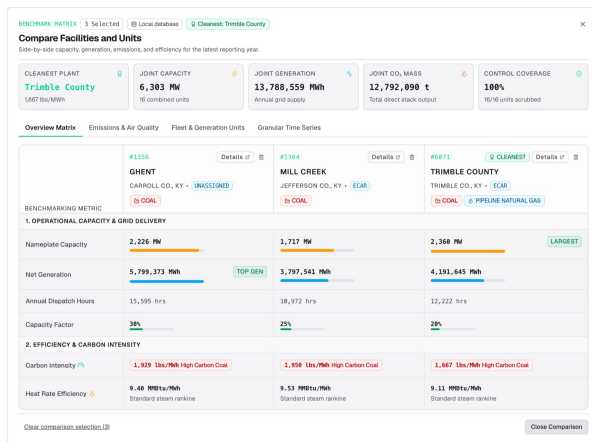


Figure 2: Comparison of three Kentucky plants.

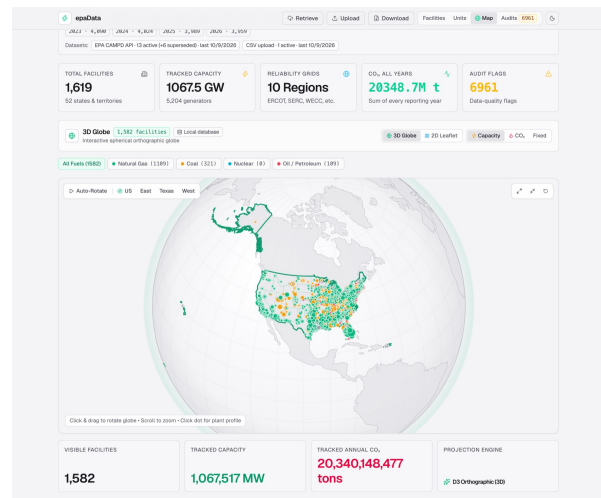


Figure 3: Map tab: globe view, coloured by fuel.

2.7 Download

The **Download** dialog (Figure 13) offers the six exports the specification requires, all as CSV: a complete dataset; its valid records; its invalid-record report (rejected and duplicate rows with their original columns, plus records flagged by a sanity rule); the current search results; a selection of facilities or units; and the provenance table, one row per dataset with source, date, parameters, and counts.

2.8 Requirement traceability

Table 1 maps each requirement of the course project description and each rubric item to the function that satisfies it.

Table 1: Requirements and where epaData meets them.

Requirement	Implementation	Status
§5 Retrieval via CAM API, key on server	Retrieve dialog; <code>CAMPD_API</code> read only on the server	Met
§5 Filters: year, state, facility, fuel, unit type, control	Retrieve parameters	Met
§5 Preserve source, date, parameters, counts, year	<code>datasets</code> row per year; history table	Met
§6 Steps 1–11 of the upload process	Upload dialog, <code>parse_import.py</code> , approval step, archive	Met
§6 Never discard invalid records silently	<code>import_issues</code> ; invalid-record export	Met
§7 Dataset, facility, unit, annual-record tables	Six tables, keys and cascades (Section 3)	Met ¹
§8.1 Basic search fields	Filter bar and <i>More filters</i>	Met
§8.2 Multi-criteria and range search	AND-combined filters, seven min/max ranges	Met
§8.3 Top- <i>N</i> , bottom- <i>N</i> , group-and-rank, comparison, history	Ranking control, compare dialog, unit dialog, year range	Met
§8.4 Readable, paginated, sortable, linked, downloadable, reproducible	Tables, pagination, unit dialog, CSV, URL state	Met
§8.4 Indexes on searched fields	Fourteen secondary and unique indexes	Met
§9 Six pages	Home view; Retrieve, Upload, detail, and Download dialogs; explorer tabs	Met
§10 Six download types, CSV	Download dialog, <code>/api/export</code>	Met
Rubric 6: description-based search	Rule-based parser, optional language-model fallback	Met

2.9 Planned functions not completed or changed

Our Phase 1 proposal listed several technologies and features that changed during development. Table 2 lists them with the reason for each change.

Table 2: Proposal versus delivered system.

Proposed	Delivered	Reason
Prisma ORM	Drizzle ORM	Drizzle’s SQL-like query builder expresses the window functions and correlated subqueries the search needs (Section 7).
Zod and Fast-CSV for upload parsing	Python script (<code>csv</code> , <code>openpyxl</code>)	The specification requires the file to be read with Python; Excel support came with it.
CSV upload only	CSV and Excel	Required by the specification.
Leaflet map with coordinate clustering	Leaflet map with scaled markers, and a D3 globe	Not completed: markers are sized and coloured instead of clustered.

¹Control and program information is stored per unit rather than per unit-year; see Section 8.

Proposed	Delivered	Reason
Anomaly detection including spikes	Three physical-sanity rules	Not completed: no year-over-year spike detection.
Large-dataset stress tests	Timing measurements on the full database (Section 6)	Not completed as a separate test suite.
Hosted deployment	Runs locally	Dropped; SQLite on serverless hosting needed workarounds (Section 8).
—	DB-vs-API preview, provenance, origin filter, description search, granular time series, globe	Added beyond the proposal.

3 System design

3.1 Architecture

epaData is a single Next.js application that contains both the user interface and the server. Figure 4 shows its components. The browser renders React components; the first page load is rendered on the server with the data already fetched, and later interactions call the server through typed tRPC procedures or two plain HTTP route handlers. On the server, every database query is built with the Drizzle ORM and executed against a single SQLite file through the libSQL client. Two kinds of work leave the Node.js process: reading uploaded files, which is delegated to a Python script run as a subprocess, and calls to external services, namely the EPA CAM API and, optionally, the OpenRouter language-model service.

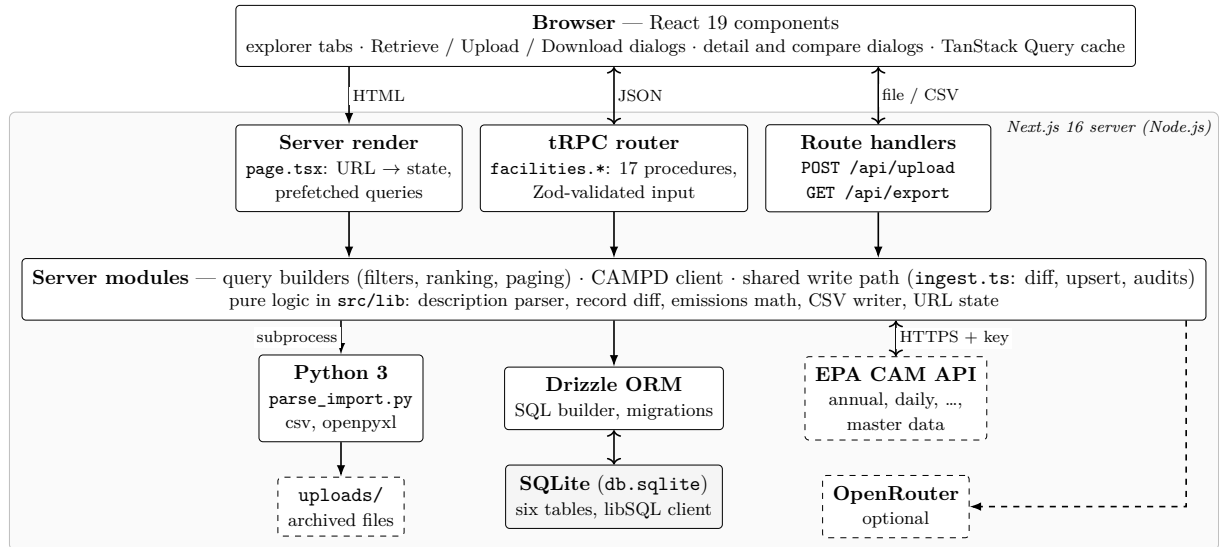


Figure 4: Architecture. Solid boxes run inside the Next.js server process; dashed boxes are external services or files. The browser never talks to the EPA API or OpenRouter directly, so their keys stay on the server.

The code is organised in layers that mirror the figure. `src/app` contains the page and its React components. `src/server/api` defines the tRPC router, whose procedures validate their input with Zod and delegate to `src/server`. That directory holds the query builders, the CAMPD client, the upload and export modules, and the shared write path. `src/lib` contains logic with no database or network access: filter and URL state, the description parser, record comparison, emissions arithmetic, and CSV formatting. Keeping this logic pure is what makes it testable

with plain unit tests (Section 6). The same modules are used by the command-line scripts in `scripts/`, so the CLI sync and the in-app retrieval run identical code.

3.2 Life cycle of a search

Following one search through the system shows how the components interact. Suppose a user opens the Units tab, chooses Kentucky and Coal, sets the year to 2025, and enters a minimum CO₂ of 500,000 tons.

1. Each change updates a filter object held in React state. A pure function, `explorerSearchParams`, mirrors the object into the URL, so the browser history and any shared link carry the full search:
`?tab=units&stateCode=KY&primaryFuel=Coal&year=2025&co2MassTonsMin=500000`
2. A deferred copy of the filters is passed to `api.facilities.getUnitYears.useQuery`. Deferring lets typing stay responsive; TanStack Query keeps the previous page on screen until the new one arrives and caches each distinct query.
3. On the server, Zod validates the input against `facilityFilterSchema` extended with paging and sort fields, rejecting unknown sort keys or out-of-range page sizes.
4. `rankedUnitYears` builds one SQL statement: `annual_records` joined to `units`, `facilities`, and `datasets`; a WHERE clause assembled from three condition builders (facility, unit, and record conditions); and a `ROW_NUMBER()` window over the requested sort. Listing 4 in Section 5.4 shows the generated SQL.
5. `pageOf` runs two queries over the same ranked subquery, one for the page of rows and one for the total count, and returns both. SQLite answers in about 5 ms using the `(year, co2_mass_tons)` index.
6. The table renders 25 rows across three pages. The **CSV** button links to `/api/export?type=search&...` with the same query string. That handler parses the string with the same `parseExplorerParams` function and runs the same `rankedUnitYears` without a page limit, so the file contains exactly the rows the table showed.

When the page is reloaded, `page.tsx` parses the URL on the server, prefetches the same query, and sends the rendered table with the data embedded, so a shared link opens directly on the results.

3.3 Database schema

Figure 5 shows the six tables and their relationships. Following the relational model [4], each table holds one kind of entity, identified by a key, and relationships are expressed through foreign keys. The design follows the real-world hierarchy of the data: a *facility* (a power plant, identified by its EPA ORISPL code) houses one or more *units* (boilers, turbines, or engines), and each unit reports one *annual record* per year. Two tables record where data came from: *datasets* has one row per retrieval or upload, and *import_issues* has one row per source row that was rejected or skipped. A sixth table, *data_audit_logs*, holds the physical-sanity flags raised on annual records.

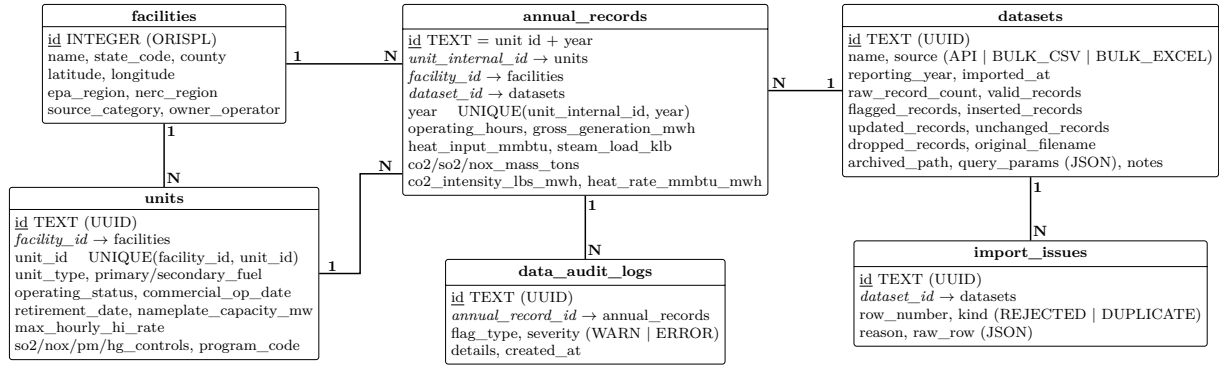


Figure 5: Entity-relationship diagram. Underlined columns are primary keys; italic columns are foreign keys (arrows name the referenced table). All foreign keys cascade on delete.

Keys and integrity rules. Each table enforces the rules of its domain in the database rather than only in application code:

- **facilities.id** is the EPA facility ID itself. The identifier is stable and meaningful, so it serves directly as the primary key, and URLs and exports can use it.
- A unit is identified within its facility by EPA’s unit ID (“1”, “CT1”, ...), which is not unique across facilities. The table therefore has a surrogate UUID key and a unique index on (*facility_id*, *unit_id*); upserts use that pair as their conflict target.
- An annual record is unique per (*unit_internal_id*, *year*), the “facility-unit-year” of the specification. Its primary key is derived from the same pair (*{unitInternalId}_-{year}*), so re-importing a year updates the existing row instead of inserting a duplicate.
- Metric columns are **REAL NOT NULL DEFAULT 0**. Missing CAMPD values become 0, so sums and ranges never meet **NULL**. The two derived rates are nullable, because they are undefined when gross load is zero.
- Foreign keys cascade on delete: deleting a facility removes its units and records, and deleting a dataset removes its issues and the records it last wrote.

Indexes. Table 3 lists the indexes and the searches they serve. One of them, (*facility_id*, *year*), was added after profiling; Section 6 explains why.

Table 3: Secondary and unique indexes.

Index	Serves
<code>facilities(state_code), (name), (county), (nerc_region), (source_category)</code>	Basic-search filters and name sorting
<code>units(facility_id, unit_id) UNIQUE</code>	Unit identity; upsert target
<code>units(facility_id), (primary_fuel), (operating_status)</code>	Joins and unit filters
<code>annual_records(unit_internal_id, year) UNIQUE</code>	Facility-unit-year identity; upsert target; unit history
<code>annual_records(year, co2_mass_tons)</code>	Year filter and CO ₂ ranking
<code>annual_records(facility_id, year)</code>	Per-facility totals, overall or within one year
<code>data_audit_logs(annual_record_id), import_issues(dataset_id)</code>	Flags per record; issues per dataset

Normalisation and one deliberate deviation. The schema is in third normal form with one exception: `annual_records.facility_id` repeats information derivable through `units`. The redundancy lets per-facility aggregates read a single indexed table instead of joining through `units` for each of the 1,619 facilities. The specification places SO₂, NO_x, and PM control information and the program code on the annual record; `epaData` stores them on the unit, because `CAMPD` reports them per unit and they rarely change from year to year. Section 8 discusses the consequence.

3.4 Data flow and provenance

Figure 6 shows how data enters and leaves the database. There are three ways in, all converging on one write path in `src/server/ingest.ts`: the Retrieve dialog, the Upload dialog, and the `sync:campd` command-line script. Before writing, the write path loads the stored metrics of every incoming facility-unit-year and classifies each record as new, changed, or unchanged. It then upserts facilities, units, and annual records, computes the two derived rates, and replaces the record’s sanity flags. Finally it records the counts on the dataset row. Source rows that fail validation or repeat a facility-unit-year are written to `import_issues` with their original values and a reason.

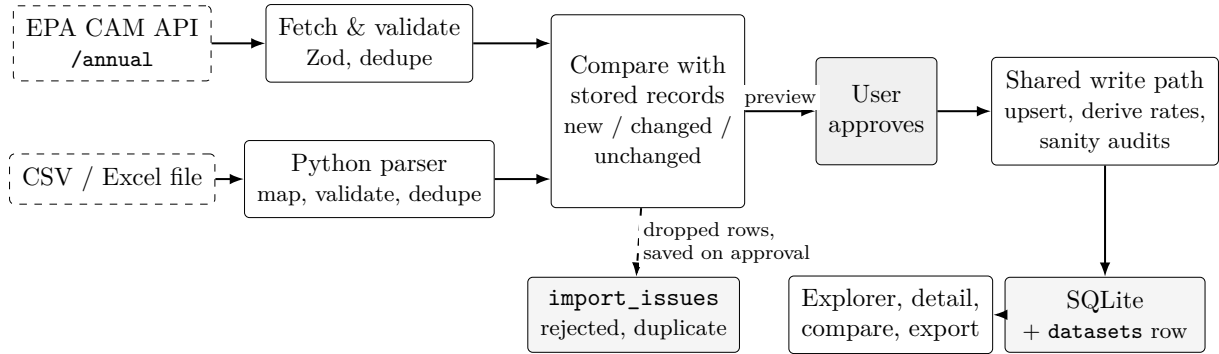


Figure 6: Data flow. Retrieval and upload differ only in how rows are obtained and validated; comparison, approval, writing, and issue logging are shared. Nothing is written before approval. The `sync:campd` script skips the approval step.

The provenance model has three levels of detail. At the record level it is a coarse form of what Buneman et al. call *where-provenance* [3]: for each stored record, it identifies the source from which the record was copied. Each *dataset* records its source, time, parameters (for API retrievals, the exact query as JSON), file name and archived copy (for uploads), and the counts of rows received, stored, flagged, new, updated, unchanged, and dropped. Each *annual record* points to the dataset that last wrote it, which the interface shows as the record’s origin. Each *issue* keeps a rejected or duplicate source row verbatim. When a later retrieval takes over all records of an earlier one, the earlier dataset is kept and labelled *superseded*, so the retrieval history is never rewritten.

4 Technical details

4.1 Languages

Three languages are used, each where it fits best. **TypeScript** is used for the user interface, the server, the query builders, and the command-line scripts. A single typed language across browser and server lets the tRPC router’s input and output types flow into the React components without hand-written API definitions. The compiler then rejects a call that passes a misspelled filter or reads a field the server does not return. **SQL** is the language of the database; most of it is generated by Drizzle, but window functions, correlated subqueries, and the provenance conditions are written as SQL fragments embedded in TypeScript. **Python** reads uploaded files, as the specification requires, using the standard `csv` module and `openpyxl` for Excel workbooks.

4.2 Database system

The database is **SQLite** [6], accessed through the libSQL client (`@libsql/client`). SQLite stores the whole database in one file, `db.sqlite`, which is committed to the repository so that the application runs immediately after installation. SQLite supports everything the design needs: foreign keys with cascading deletes, unique and composite indexes, upserts (`INSERT ... ON CONFLICT DO UPDATE`), and window functions such as `ROW_NUMBER() OVER (PARTITION BY ...)`, which implement ranking. Window functions let the database rank rows within groups in a single pass over sorted input [8], instead of self-joins or ranking in application code. The libSQL client would also allow the same code to connect to a hosted libSQL server by changing `DATABASE_URL`, although the delivered system uses only the local file.

4.3 Frameworks and libraries

Table 4 lists the frameworks and libraries and the role each plays.

Table 4: Frameworks and libraries.

Component	Role in epaData
Next.js 16 (App Router)	Application framework: server rendering of the page, route handlers for upload and export, bundling, and the development and production servers.
React 19	User-interface components: tables, filter bar, dialogs.
tRPC 11	Typed remote procedure calls between browser and server; the 17 <code>facilities.*</code> procedures.
TanStack Query 5	Client-side caching, background refetching, and loading states for tRPC calls.
Drizzle ORM and drizzle-kit	Schema definition in TypeScript, type-checked query building, and generated SQL migrations (six so far, in <code>drizzle/</code>).
Zod	Validation of every procedure input, of CAMPD API rows, of environment variables, and of the optional language model’s output.
Python 3, csv, openpyxl	Reading, mapping, and validating uploaded CSV and Excel files.
Tailwind CSS 4	Styling with design tokens for light and dark themes.
Leaflet	Two-dimensional map with CARTO/OpenStreetMap tiles.
D3 [2] and TopoJSON	Orthographic globe drawn on a canvas from bundled U.S. and world outlines.
lucide-react, next-themes	Icons; theme switching.
<code>node:test</code> with tsx	Unit-test runner for TypeScript without a separate test framework.
ESLint, Prettier,	Static checks run by <code>npm run validate</code> .
TypeScript compiler	
Tectonic	Builds this report from LaTeX.

4.4 External services

The **EPA CAM API** provides apportioned emissions at annual, monthly, daily, and hourly resolution, plus master-data endpoints listing valid fuel types, unit types, and control technologies. Requests carry the API key in an `x-api-key` header, are made only from the server, and are paged at 500 rows per request. A safety cap of 100 pages raises an error instead of silently truncating a result. **OpenRouter** is an optional gateway to hosted language models, used only as a fallback by the description search (Section 5.3); without a key, the feature works with the rule-based parser alone.

4.5 Scripts

The npm scripts in `package.json` are the entry points for setup, data maintenance, and quality checks (Table 5). The three data scripts in `scripts/` import the same server modules as the web application, so a command-line sync and an in-app retrieval run the same code.

Table 5: Scripts.

Command	Purpose
<code>npm run dev / build / start</code>	Development server; production build and server.
<code>npm run db:migrate</code>	Applies the SQL migrations in <code>drizzle/</code> to the database named by <code>DATABASE_URL</code> .
<code>npm run db:generate</code>	Generates a migration after a change to <code>src/server/db/schema.ts</code> .
<code>npm run db:seed</code>	Loads facilities and units from CAMPD facility CSV files.
<code>npm run sync:campd</code>	Retrieves annual emissions for one year or a range and stores them (<code>--year</code> , <code>--from</code> , <code>--to</code>).
<code>npm test</code>	Runs the 39 unit tests in <code>src/lib/*.test.ts</code> .
<code>npm run validate</code>	Format check, lint, type check, tests, and production build.
<code>npm run report</code>	Builds this report (<code>report/main.pdf</code>).
<code>scripts/parse_import.py</code> FILE	Prints the validation report of a CSV or Excel file as JSON; called by the upload endpoint.

4.6 Configuration

Configuration is read from environment variables defined in `.env`, validated at start-up by `src/env.js`, and documented in `.env.example`. Only `DATABASE_URL` is required. `CAMPD_API` enables retrieval and the granular views, and `OPENROUTER_API_KEY` and `OPENROUTER_MODEL` enable the language-model fallback. `NEXT_PUBLIC_CARTO_API` improves the map tiles. Variables without the `NEXT_PUBLIC_` prefix are never included in the browser bundle, which is how the specification’s rule that the EPA key must not reach users is enforced. The README gives the complete installation procedure.

5 Highlighted features

This section presents the six features that most distinguish `epaData`, each with screenshots and an example workflow. All screenshots were taken from the running application on a copy of the committed database.

5.1 Comparing the database with the EPA API before saving

Most data loaders write whatever the source returns. `epaData` treats retrieval like a merge: it shows what a retrieval *would* change and saves only after the user agrees. This was the most requested capability during development, because the main question a user has before pressing “retrieve” is whether anything is new.

Example workflow. The committed sample upload (Section 5.2) lowers one stored value: the 2023 CO₂ mass of Trimble County unit 2, from 4,099,943.863 to 4,098,943.863 short tons. After that upload, the user opens Retrieve and chooses Kentucky, facility 6071, and 2023:

1. While the form is edited, *Already in the database* reports eight stored unit-years for 2023: two last written by the CSV upload, six by an earlier API retrieval.
2. *Preview* fetches the year from the API and compares each row with the stored record. Figure 7 shows the result: eight rows received, no new rows, one changed, seven unchanged, none dropped. The Changed tab names the field and both values, “CO₂: 4,098,943.86 → 4,099,943.86”.

3. *Approve & save* stores the year as a new dataset. The history records 0 new, 1 updated, 7 unchanged, 0 dropped, and the EPA value is restored.

CAMPD Retrieval

×

Retrieve EPA CAMPD Annual Emissions

Method: EPA Clean Air Markets (CAM) API, apportioned annual emissions (/annual). Preview compares what the API returns with what the database already holds; nothing is saved until you approve. Each year is stored as its own dataset. Files from CAMPD Custom Data Download go through Upload instead.

PARAMETERS

FROM YEAR

TO YEAR

FACILITY IDS

STATE

FUEL

UNIT TYPE

CONTROL TECHNOLOGY

ALREADY IN THE DATABASE

Stored unit-years for these years and filters, by the dataset that last wrote them. Fuel, unit type, and control match by name, so this approximates what CAMPD will return.

Local database

YEAR	STORED UNIT-YEARS	LAST WRITTEN BY
2023	2	CSV upload - 10/9/2026
2023	6	EPA CAMPD API - 10/6/2026

PREVIEW: WHAT THE API RETURNED

Compared with the stored values for each facility-unit-year. Approving writes new and changed records; unchanged ones are rewritten with the same values, and dropped rows are listed in the invalid-records report.

Live EPA API

RECEIVED

NEW

CHANGED

UNCHANGED

DROPPED

New

Changed

Unchanged

Dropped

YEAR	FACILITY	UNIT	GROSS MWH (API)	CO ₂ T (API)	DATABASE → API
2023	Trimble County (#6071)	2	4,614,178	4,899,944	CO ₂ : 4,098,943.86 → 4,099,943.86

Up to 50 rows per category and year are listed; the counts above cover every row.

PAST RETRIEVALS

New / Updated / Unchanged compare each import with what the database held before it (— = imported before this was tracked). Superseded datasets had all their records taken over by a later retrieval; they are kept as history.

YEAR	SOURCE	PARAMETERS	IMPORTED	RECEIVED	STORED	NEW	UPDATED	UNCHANGED	DROPPED
------	--------	------------	----------	----------	--------	-----	---------	-----------	---------

Cancel

Approve & save year (1 new or changed)

Close

Figure 7: Retrieval preview for Trimble County, 2023: the database panel (top), the API comparison with one changed record and its old and new value, and the approval button.

How it works. Preview and save share one fetch function, `fetchCampdAnnual`, which pages through the API and validates every row with Zod. It also removes rows that repeat a facility-unit-year, also across pages, and writes nothing. The comparison keys every record by its natural key, facility ID, unit ID, and year, and compares the seven stored metrics (Listing 1). Values that differ only by floating-point noise count as equal, so re-reading the same CAMPD figure never shows as a change. Rows that fail validation or repeat a key are *dropped*, but never silently: saving writes them to `import_issues` with the Zod message as the reason, so the invalid-record report covers API datasets as well as uploads. The save step fetches the year again rather than trusting the preview, and the whole year is fetched before the first write. A network failure part-way through therefore leaves no partial dataset.

```

1 /** Natural key of a unit-year record, shared by uploads, CAMPD rows, and stored records. */
2 export const recordKey = (facilityId: number, unitId: string, year: number | string) =>
3   `${facilityId}:${unitId}:${year}`;
4
5 /** Equal within floating-point noise (relative 1e-9). */
6 const same = (a: number, b: number) =>
7   Math.abs(a - b) <= 1e-9 * Math.max(1, Math.abs(a), Math.abs(b));
8
9 /** New when nothing is stored; otherwise every metric that differs from the stored value. */
10 export function diffRecord(incoming: EmissionTotals, stored?: EmissionTotals) {

```

```

11  if (!stored) return { status: "new", changes: [] };
12  const changes = TOTAL_KEYS.filter((k) => !same(incoming[k], stored[k])).map(
13    (field) => ({ field, database: stored[field], incoming: incoming[field] }),
14  );
15  return { status: changes.length ? "changed" : "unchanged", changes };
16  }

```

Listing 1: Record comparison (src/lib/record-diff.ts), shared by retrievals and uploads.

5.2 Upload validation and the data-quality report

The upload path implements all eleven steps of the specification. Its distinguishing property is that nothing is lost: every row ends up either stored, or stored and flagged, or listed in the data-quality report with a reason.

Validation rules. scripts/parse_import.py normalises each header by lower-casing it and removing punctuation, so “Facility ID”, facility_id, and facilityId all match the same alias list. It then maps the file to one of two schemas: annual emissions (facility ID, unit ID, year, and metrics) or facilities and units (no year). Each cell is converted by a column-specific rule (Listing 2). Identifiers and years must be whole numbers within bounds. Coordinates must lie within $\pm 90/\pm 180$ degrees. Metrics must be non-negative numbers, and states must be valid U.S. state or territory codes. A row with any failing cell is rejected with every reason listed. A row whose facility-unit-year already appeared earlier in the file is a duplicate and is skipped with a reference to the first occurrence.

```

1  def parse_value(col, raw):
2      """Convert a non-empty cell to its database value, or raise ValueError with the reason.
3      """
4      if col == "stateCode":
5          if raw.upper() not in US_STATES:
6              raise ValueError(f"Unrecognized US state or territory '{raw}'")
7          return raw.upper()
8      if col not in BOUNDS:
9          return raw
10     lo, hi, whole = BOUNDS[col]          # e.g. year: (1980, 2100, True)
11     try:
12         n = float(raw.replace(",", ""))
13     except ValueError:
14         n = math.nan
15     if not math.isfinite(n):
16         raise ValueError(f"Expected a number, got '{raw}'")
17     if whole and not n.is_integer():
18         raise ValueError(f"Expected a whole number, got '{raw}'")
19     if n < lo or (hi is not None and n > hi):
20         raise ValueError(f"Out of range ({lo} to {hi})" if hi is not None else f"Must be at least {lo}")
21     return int(n) if whole else n

```

Listing 2: Cell validation in scripts/parse_import.py (abridged).

Example workflow. The repository ships a test file, samples/annual-emissions-sample.csv, with an identical Excel copy. It contains 25 rows of real CAMPD data for Kentucky, mostly from 2014, a year the committed database does not cover. Five rows were altered on purpose, and one unaltered row trips a sanity rule. Uploading it produces the report in Figure 8:

- 21 valid rows: 19 new, one changed, one unchanged;
- three rejected rows: a missing unit ID (row 12), the state code “KZ” (row 18), and a negative CO₂ mass (row 19);
- one duplicate: row 25 repeats row 4;
- one sanity flag: row 16, Robert Reid unit RT in 2014, which reported 38,801.9 MMBtu of heat input and no CO₂. This row is unmodified EPA data.

After approval, the Units tab filtered to origin “File upload” lists exactly the 21 stored unit-years, and the invalid-record download contains the three rejected rows, the duplicate, and the flagged record, each with its original columns.

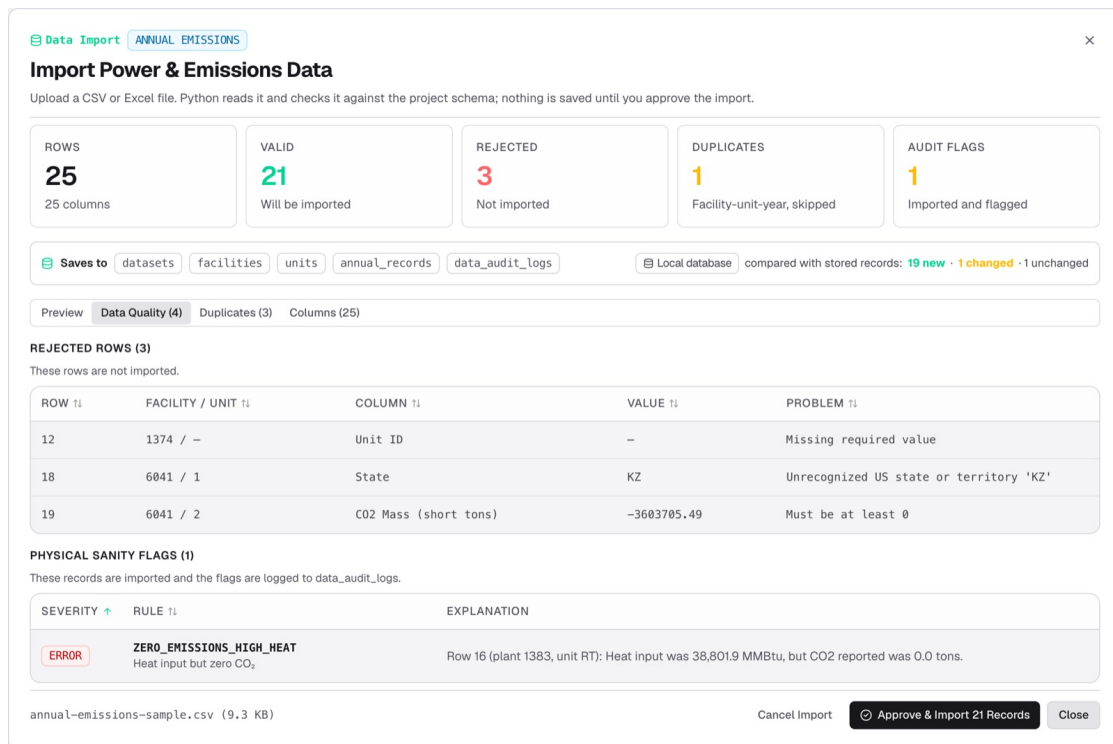


Figure 8: Upload report for the sample file, Data Quality tab: counts, the database comparison, the three rejected rows with their reasons, and the physical-sanity flag.

5.3 Description-based search

The rubric asks for a search that understands requests such as “Find coal units in Kentucky with high CO₂ emissions.” A large language model could translate such a sentence, but it would make the core search depend on a network service and an API key, and its output can differ from run to run. epaData instead uses a deterministic, rule-based parser (`src/lib/describe-search.ts`). A language model is consulted only for words the parser could not use, and only if a key is configured.

Algorithm. The parser performs entity extraction rather than matching sentence templates, so word order does not matter. It runs in two passes.

```
vocabulary <- filter values from the database (states, fuels, unit types, controls,
  counties, statuses) + synonyms ("scrubber" -> FGD, "gas" -> Natural Gas,
  state names -> codes) + metric, comparator, ranking, and view words
```

```

pass 1: tokens -> items
  for each position, take the LONGEST phrase (up to 6 words) found in the vocabulary
    ("west virginia" beats "virginia"; "natural gas" beats "gas")
  two-letter state codes count only in capitals ("IN", not "in")
  counties need the word "county"; names follow "owned by" / "named"
  numbers: 500,000 | 500k | 1.2M | 5e5; four-digit years
  unknown filter value within edit distance 1 (2 for long words) -> fuzzy match
  "not X" -> reported as not understood, never inverted
  anything else that is not a stopword -> not understood

pass 2: bind items
  filters -> filter values; "units" / "plants" -> which table to show
  comparator + number -> nearest metric word, looking forward then back
    ("over 500k tons of CO2", "CO2 over 500k", "under 50 tons SO2")
  year-like numbers -> year, or year range ("since 2020", "2015-2020")
  "top / highest / most [N]" -> sort descending, keep N (default 10, or 1 if singular)
  "lowest / least / cleanest [N]" -> sort ascending; "in each state" -> rank per state
  "high / low <metric>" without a number -> qualitative term, resolved on the server

```

Listing 3: Outline of the description parser.

Typo tolerance uses the optimal-string-alignment variant of edit distance, which counts insertions, deletions, substitutions, and transpositions of adjacent letters, the four error types Damerau identified as the bulk of spelling mistakes [5]. It applies only to filter values of five letters or more, so short words are never “corrected” into something else.

Resolving “high” and “low”. A qualitative term cannot be turned into a fixed number in advance: 500,000 tons of CO₂ is high for a gas turbine and low for a large coal unit. The server therefore resolves “high m ” to the 75th percentile, and “low m ” to the 25th percentile, of metric m over the unit-years that match all the *other* filters. With n matching values sorted in ascending order, $x_{(0)} \leq \dots \leq x_{(n-1)}$, the threshold is the order statistic

$$t_{\text{high}} = x_{(\lfloor 0.75(n-1) \rfloor)}, \quad t_{\text{low}} = x_{(\lfloor 0.25(n-1) \rfloor)}, \quad (1)$$

obtained with a single `ORDER BY  $m$  LIMIT 1 OFFSET  $k$` query. This is the simplest of the sample-quantile definitions compared by Hyndman and Fan [7]: where their common default interpolates between the two neighbouring order statistics, epaData takes the lower one, so the threshold is always a value that occurs in the data. The threshold is rounded to three significant figures and becomes an ordinary minimum or maximum filter, and the table is sorted by that metric. For the rubric sentence, the 416 Kentucky coal unit-years give $t_{\text{high}} = 2,660,000$ tons, which selects 103 unit-years (Figure 9). The interface states the rule next to the result, “high = top 25% of unit-years matching the other filters”.

FACILITY	STATE	UNIT	YEAR	ORIGIN	MW	OP. HRS	GROSS MWH	HEAT MMBTU	CO ₂ T	SO ₂ T	NO _x T
Trimble County #6871	KY	Unit 2 Dry bottom wall-fired boiler	2015	API	750	7,945	5,836,029	50,793,165	5,327,182	1,590.3	941
Paradise #1378	KY	Unit 3 Cyclone boiler	2016	API	1,150.2	6,390	5,339,177	51,126,130	5,245,537	2,990.2	3,832
Trimble County #6871	KY	Unit 2 Dry bottom wall-fired boiler	2021	API	750	7,916	5,745,036	47,116,891	4,941,616	1,290.2	726
East Bend #6818	KY	Unit 2 Dry bottom wall-fired boiler	2017	API	669.3	7,911	4,638,971	47,715,141	4,895,572	2,630.2	2,886
Paradise #1378	KY	Unit 3 Cyclone boiler	2018	API	1,150.2	5,304	4,303,206	45,954,609	4,714,942	2,538.7	4,305
Trimble County #6871	KY	Unit 2 Dry bottom wall-fired boiler	2017	API	750	7,077	5,128,592	44,925,315	4,711,755	1,386.8	987
Trimble County #6871	KY	Unit 2 Dry bottom wall-fired boiler	2020	API	750	7,586	5,305,838	44,755,622	4,693,968	1,835.5	846
Trimble County #6871	KY	Unit 2 Dry bottom wall-fired boiler	2022	API	750	7,649	5,274,842	44,748,950	4,693,267	1,436.4	766
East Bend #6818	KY	Unit 2 Dry bottom wall-fired boiler	2015	API	669.3	7,796	4,411,252	45,601,205	4,678,681	2,656.8	5,982
Paradise #1378	KY	Unit 2 Cyclone boiler	2015	API	704	7,639	4,278,547	44,862,235	4,602,866	7,309.4	2,345

Figure 9: Description search on the committed database. The sentence becomes three removable filter chips; the note explains the threshold; 103 unit-years match, sorted by CO₂.

Language-model fallback. When words remain unrecognised and `OPENROUTER_API_KEY` is set, `src/server/llm-search.ts` sends the sentence, the parser’s interpretation, and the leftover words to an OpenRouter model. The request asks for structured JSON whose schema enumerates the database’s own filter values. The model may only *add* filters the parser did not set. Every returned value is re-validated with Zod against the vocabulary, the call times out after 8 seconds, and any failure falls back to the parser’s result with a note. The key never leaves the server. The language model therefore cannot introduce a value the database does not contain, and the search works fully without it.

Worked examples. Table 6 lists ten descriptions, the filters the parser produced, and the number of rows each returns on the committed database. None of them needed the language model.

Table 6: Description search on the committed database (parser only).

Description	Interpreted as	Rows
Find coal units in Kentucky with high CO2 emissions.	Units; KY; Coal; CO ₂ ≥ 2,660,000; sort CO ₂ ↓	103
coal units in Kentucky in 2025 with CO2 over 500,000 tons	Units; KY; Coal; 2025; CO ₂ ≥ 500,000	25
top CO2-emitting facility in each state in 2024	Facilities; 2024; first 1 per state by CO ₂ ↓	51

Description	Interpreted as	Rows
top 20 units by gross generation in 2025	Units; 2025; first 20 by gross load↓	20
units with the lowest NOx emissions in 2025	Units; 2025; first 10 by NO _x ↑	10
gas units in TX under 50 tons SO ₂	Units; TX; Natural Gas; SO ₂ ≤ 50	4,374
operating gas units in Texas with low NOx since 2020	Units; TX; Natural Gas; Operating; year ≥ 2020; NO _x ≤ 8.46; sort NO _x ↑	670
plants owned by Duke Energy with scrubbers	Facilities; text “Duke Energy”; SO ₂ control FGD	11
units in IN with heat input above 5M MMBtu in 2025	Units; IN; 2025; heat input ≥ 5,000,000; sort heat input↓	42
Kentucky coal units that are not retired	Units; KY (typo corrected); Coal; <i>not understood: “not retired”</i>	416

5.4 Multi-constraint search and ranking in SQL

The specification’s example, “coal-fired units in Kentucky in 2025 with annual CO₂ emissions greater than 500,000 short tons”, is a conjunction of three kinds of condition: on the facility (state), on the unit (fuel), and on the unit-year (year and CO₂). The query builder keeps one function per kind and combines their output with AND (Figure 10). Fuel, unit-type, and control filters match by substring, because CAMPD stores combined values such as “Coal, Pipeline Natural Gas” or “Wet Lime FGD|Wet Limestone”. Operating status matches by prefix, so that “Retired” does not also match “Operating (Retired 04/15/2016)”.

epaData Retrieve Upload Download Facilities Units Map Audits 6961

1,619 52 states & territories **1067.5 GW** 5,204 generators **10 Regions** ERCOT, SERC, WECC, etc. **20348.7M t** Sum of every reporting year **6961** Data-quality flags

Name Describe

KY All Grids Coal Reset More filters **25** unit-years Local database - last import 10/9/2026 CSV

Filters: State: KY x Fuel: Coal x Year: 2025 x CO₂ ≥ 500,000 tons x

FACILITY ID: UNIT ID: REPORTING YEAR: COUNTY: SECONDARY FUEL:

UNIT TYPE: SO₂ CONTROL: NO_x CONTROL: PM CONTROL: OPERATING STATUS:

AUDIT FLAG: FLAG SEVERITY: ORIGIN:

RANGES PER UNIT-YEAR (MIN / MAX, INCLUSIVE)

Reporting year (yr): Operating time (hr): Gross load (MWh):

Heat input (MMBtu): CO₂ (tons): SO₂ (tons):

NO_x (tons):

RANKING: GROUP: Ranks follow the table sort: click a column header (↓ = Top-N, ↑ = Bottom-N). Each row is one unit in one reporting year.

	FACILITY T1	STATE T1	UNIT T1	YEAR T1	ORIGIN	MW T1	OP. HRS T1	GROSS MWH T1	HEAT MMBTU T1	CO ₂ T T1	SO ₂ T T1	NO _x T T1
☐	Trimble County #6871	KY	Unit 2 Dry bottom wall-fired boiler Coal	2025	API	750	7,053	4,888,516	40,531,843	4,250,980	1,494	777.4
☐	H L Spurlock #6841	KY	Unit 2 Tangentially-fired Coal	2025	API	610	7,374	3,465,337	35,746,445	3,667,586	1,329.3	1,534.3
☐	Trimble County #6871	KY	Unit 1 Tangentially-fired Coal	2025	API	566.1	7,711	3,758,564	33,145,831	3,400,682	2,619.7	1,187.6
☐	Mill Creek #1364	KY	Unit 4 Dry bottom wall-fired boiler Coal	2025	API	543.6	7,257	3,135,113	30,086,675	3,083,035	792.3	1,356.5
☐	Ghent	KY	Unit 2 Tangentially-fired	2025	API	556.4	8,705	3,716,507	30,045,786	3,877,470	5,660.5	2,667.0

Figure 10: Multi-constraint search: KY, Coal, 2025, CO₂ ≥ 500,000 t. The chips, the open *More filters* panel with its range inputs, and the first rows of the 25 results.

Ranking is implemented with a window function. Listing 4 shows, simplified, the SQL that `rankedUnitYears` generates for “the highest-CO₂ coal unit-year in each state in 2025”. The inner query numbers the matching rows in sort order, restarting the numbering for each state when ranking per state. The outer query keeps the first *N* ranks. The same subquery feeds the page of rows, the total count, and the CSV export, so all three always agree. IS NULL in the ordering puts missing values last in either direction, and the record ID breaks ties so that paging is stable.

```

1 SELECT * FROM (
2   SELECT ar.id, f.name, f.state_code, u.unit_id, ar.year, ar.co2_mass_tons, d.source AS
      origin,
3     ROW_NUMBER() OVER (
4       PARTITION BY f.state_code                                -- only when ranking per state
5       ORDER BY ar.co2_mass_tons IS NULL, ar.co2_mass_tons DESC, ar.id
6     ) AS rank
7 FROM annual_records ar
8 JOIN units u      ON ar.unit_internal_id = u.id
9 JOIN facilities f  ON ar.facility_id = f.id
10 LEFT JOIN datasets d ON ar.dataset_id = d.id
11 WHERE lower(u.primary_fuel) LIKE '%coal%'                    -- unit condition (contains)
12    AND ar.year = 2025                                         -- record conditions
13 ) AS ranked
14 WHERE rank <= 1                                              -- keep the first N
15 ORDER BY state_code, rank
16 LIMIT 25 OFFSET 0;                                         -- one page

```

Listing 4: Ranking query generated for a per-state top-1 search (simplified).

The Facilities view ranks plants by totals computed with correlated subqueries, for example the sum of CO₂ over a facility’s records in the selected year. Unit and unit-year filters keep a facility if at least one of its unit-years matches, expressed as an IN subquery so that the facility row is not duplicated. Figure 11 shows the per-state ranking for 2024.

Figure 11: Group-and-rank: the top CO₂-emitting facility in each state in 2024 (51 rows).

5.5 Physical-sanity audits

Validation catches values that are malformed; it cannot catch values that are well-formed but physically implausible. epaData therefore applies three rules to every annual record it writes, whatever its source. Let H be heat input (MMBtu), G gross load (MWh), T operating time (hours), and C the CO₂ mass (short tons). The derived rates stored with each record are

$$\text{heat rate} = \frac{H}{G} \text{ MMBtu/MWh}, \quad \text{CO}_2 \text{ intensity} = \frac{2000 C}{G} \text{ lb/MWh}, \quad (2)$$

both undefined (stored as NULL) when $G = 0$. The rules, with thresholds defined once in `AUDIT_THRESHOLDS`, are:

$$\text{ZERO_EMISSIONS_HIGH_HEAT (ERROR)} : H > 1000 \wedge C = 0, \quad (3)$$

$$\text{PHANTOM_GENERATION (ERROR)} : G > 0 \wedge T = 0, \quad (4)$$

$$\text{EXTREME_HEAT_RATE (WARN)} : H/G < 5 \vee H/G > 25. \quad (5)$$

The first rule reflects that burning fossil fuel always produces CO₂. The second, that a unit cannot generate without running. The third bounds the heat rate by the thermodynamic range of real fossil plants, which typically lie between about 6.5 MMBtu/MWh (modern combined-cycle units) and 15–18 MMBtu/MWh (old peaking units). Flagged records are kept, not discarded: they appear in the Audits tab, in the unit and facility dialogs, and in the invalid-record export. In the committed database the rules flag 6,960 records: 855 extreme heat rates (WARN) and 6,105 zero-CO₂ records (ERROR). Most of the latter are gas- and oil-fired units, and 2,637 of them also report no gross load. In these records CAMPD gives heat input but no CO₂ mass, which epaData stores as zero. The rule therefore cannot tell a CO₂ value that was not reported from one reported as zero, and many of these flags mean “CO₂ not reported” rather than a measurement error (Section 8).

5.6 Provenance and export

Every number in epaData can be traced. The unit dialog (Figure 12) lists each reporting year with the dataset that wrote it. The Units table shows the same information as an *Origin* badge, and the *Origin* filter restricts any search to API or uploaded records. The Download dialog (Figure 13) exports the provenance table itself: one row per dataset with its source, date, query parameters as JSON, file name, and the received, stored, flagged, new, updated, unchanged, and dropped counts. Unit-year exports include `dataset_id`, `origin`, and `dataset_imported_at` for every row. All exports are written by one CSV function that quotes fields according to RFC 4180.

Ghent - Unit 1
Owner: Kentucky Utilities Company - Carroll Co., KY

Summary:
 CAPACITY: 556.9 MW
 GROSS LOAD 2026: 1,467,467 MWh
 CO₂, 2026: 1,467,448 t
 INTENSITY: 2,000 lbs/MWh
 HISTORY: 13 years (2014–2026)

IDENTIFICATION
 Unit type: Tangentially-fired
 Commissioned: 1974-02-19
 Nameplate capacity: 556.9 MW
 Programs: ARP, CSNOK, CSOSQ2E, CSSO201, MARS

FUEL & CONTROLS
 Primary fuel: Coal
 SO₂ controls: Wet Limestone
 PM controls: Baghouse
 Secondary fuel: —
 NO_x controls: Low NO_x Burner Technology w/ Separated OFA - Selective Catalytic Reduction
 Hg controls: Additives to Enhance PAC and Existing Equipment Performance - Untreated PAC Sorbent Injection

OPERATING DATA & EMISSIONS BY YEAR
 One row per reporting year, with the dataset each record came from.

YEAR	OP. HRS	GROSS MWH	HEAT MMBTU	STEAM KLB	CO ₂ , T	SO ₂ , T	NO _x , T	LBS/MWH	MMBTU/M
2026	4,875	1,467,467	14,382,635	0	1,467,448	686.8	423.6	2,000 lbs/MWh High Carbon Cr	9.75
2025	7,264	2,757,461	26,882,683	0	2,758,166	954.3	981.7	2,001 lbs/MWh High Carbon Cr	9.75
2024	7,846	3,133,799	30,248,939	0	3,183,548	895.6	1,009.9	1,981 lbs/MWh High Carbon Cr	9.65
2023	7,288	2,899,444	27,953,962	0	2,868,875	770.8	716.8	1,978 lbs/MWh High Carbon Cr	9.64
2022	7,366	2,937,789	28,253,249	0	2,896,784	1,122.5	758.7	1,974 lbs/MWh High Carbon Cr	9.62
2021	6,696	2,929,928	27,871,685	0	2,859,635	1,399.7	978.4	1,952 lbs/MWh High Carbon Cr	9.51
2020	6,785	2,886,821	27,986,683	0	2,863,217	1,313.7	816.5	1,984 lbs/MWh High Carbon Cr	9.67
2019	7,477	2,938,256	29,113,299	0	2,987,825	811.4	889.3	2,039 lbs/MWh High Carbon Cr	9.94

Close Unit Record

Figure 12: Unit dialog: Ghent unit 1 after the sample upload, with 13 reporting years (2014 from the upload, 2015–2026 from the API).

Download Data

Export datasets, search results, your comparison selection, data-quality reports, or provenance as CSV.

DATA
 Download your record's dataset contents, with its audit flags.

Dataset: 2023 - EPA CAMPS API - KY - Facility 6071 - Active - 10/9/2028

Format: CSV (comma-separated, UTF-8)

Download CSV Close

Figure 13: Download dialog: data type, dataset, filters, and format.

6 Testing and results

6.1 Approach

The system was tested at four levels.

1. **Static checks.** `npm run validate` runs Prettier, ESLint, the TypeScript compiler in strict mode, the unit tests, and a production build; the project was kept passing after every development stage. Type checking catches a large class of errors early, because database rows, procedure inputs, and component props share types.
2. **Unit tests.** 39 tests with Node’s built-in test runner cover the logic in `src/lib`, which has no database or network access, and the Python upload parser, which the tests run as a subprocess on generated and committed files.
3. **Acceptance tests against hand-written SQL.** For each search in the specification, the result of the application, through its HTTP API, was compared with an independent SQL query written directly against the database.
4. **End-to-end tests on database copies.** The upload, retrieval, and export flows were exercised over HTTP, and the user interface was driven through a scripted headless browser, which also produced the screenshots in this report. These runs used copies of `db.sqlite`, so the committed data stayed unchanged.

6.2 Unit tests

Table 7 summarises the unit tests by file. Each test contains several assertions; the description-search tests, for example, check about sixty phrasings, including three word orders of the same request that must yield identical filters.

Table 7: Unit tests (`npm test`: 39 tests, all passing).

File	Tests	What is checked
<code>describe-search.test.ts</code>	12	Rubric and specification sentences; word-order permutations; ranges, units of measure, and <i>between</i> ; top- N , bottom- N , and per-state ranking; years and year ranges; synonyms, controls, and statuses; state names, codes, and ambiguous lower-case codes; owner names; typos; negation and unknown words; number formats.
<code>domain-utils.test.ts</code>	13	Emissions arithmetic with missing generation; fuel metadata; control rules; reporting-period dates and EPA’s published-through quarter; yearly roll-ups; retrieval filters; explorer state round-trip through the URL and rejection of invalid parameters; option splitting; sorting.
<code>data-import.test.ts</code>	7	Extension and size checks; rejected rows, duplicates, and sanity flags on a generated file; column mapping; refusal of files without IDs; Excel reading; a real CAMPD facility file (skipped when absent); the committed sample files (Section 5.2).
<code>record-diff.test.ts</code>	4	New, unchanged (including floating-point noise), and changed records with both values; counting and natural keys.
<code>csv.test.ts</code>	3	RFC 4180 quoting of commas, quotes, and line breaks; CRLF line endings; export URLs.

6.3 Representative test cases

Table 8 lists representative acceptance and end-to-end cases. The expected value comes from an independent source, usually a hand-written SQL query or the documented contents of the sample file, and the actual value from the application. All counts refer to the committed database unless stated otherwise.

Table 8: Representative test cases.

Input	Expected (source)	Actual	Pass
Units: KY, Coal, 2025, CO ₂ ≥ 500,000	25 unit-years (SQL)	25	✓
Facilities: same filters	8 facilities (SQL)	8	✓
Facilities: 2024, top 1 per state by CO ₂	51, one per state with 2024 data (SQL)	51	✓
Units: facility 3, unit 1, 2015–2025	11 years (SQL)	11	✓
Units: operating status “Retired”	31, prefix match (SQL)	31	✓
Describe: “Find coal units in Kentucky with high CO ₂ emissions.”	P75 of 416 values = 2,656,019 → 2,660,000; 103 rows (SQL)	2,660,000; 103 rows	✓
Describe: same request in three word orders	Identical filters	Identical	✓
Describe: “Kentucky coal units that are not retired”	KY, Coal; “not retired” reported	As expected	✓
CSV export of the description search	103 data rows	103	✓
2025 dataset (CAMPD API)	3,989 records, 420 flagged (SQL)	3,989; 420	✓
Upload the sample file (samples/ , CSV)	25 rows: 21 valid (19 new, 1 changed, 1 unchanged), 3 rejected, 1 duplicate, 1 flag (sample README)	As expected	✓
Upload the Excel copy	Same report as the CSV	Same	✓
Approve the sample upload	21 records stored; 4 issues; Origin = Upload lists 21	21; 4; 21	✓
Upload the sample again	21 unchanged, 0 new	21; 0	✓
Retrieve KY, facility 6071, 2023 after the upload	8 received, 1 changed (CO ₂ restored), 7 unchanged	As expected	✓
Retrieve preview, then cancel	No dataset written	Dataset count unchanged	✓
Upload a .pdf file / an empty file	Rejected before parsing	Rejected with reason	✓
Search the client and server bundles for the EPA key	0 occurrences	0	✓

6.4 Performance

Response times were measured on a laptop (Apple silicon) against a production build, as the median of three requests after one warm-up. Table 9 shows the results.

Table 9: Measured response times on the full database (50,947 unit-years).

Operation	Time
Multi-constraint unit search (KY, Coal, 2025, CO ₂ ≥ 500,000), one page	5 ms
Description search (parse and percentile threshold)	12 ms
Facilities, default view, one page	60 ms
Units, no filters (count over all 50,947 rows), one page	210 ms
Top facility per state in 2024, <i>before</i> the (facility_id , year) index	9,500 ms
Top facility per state in 2024, <i>after</i> the index	22 ms
CSV export of all 50,947 unit-years	1.4 s
Preview of a full year from the EPA API (3,989 rows, 8 pages)	2.3 s

The two ranking rows record a defect found during testing for this report. Any facility-level query with a year filter took 4–10 seconds, while the same query without a year took 90 ms. SQLite’s query plan showed why: the per-facility totals are correlated subqueries of the form

WHERE `facility_id = ? AND year = ?`, and with only single-column statistics the planner chose the `(year, co2_mass_tons)` index. It thus scanned all four thousand records of the year once for each of the 1,619 facilities. A composite index on `(facility_id, year)`, added as migration 0005, lets the planner match both conditions; the query became over four hundred times faster with identical results.

6.5 Known issues

- **Missing CO₂ is stored as zero.** CAMPD leaves CO₂ mass empty for some records that do report heat input. Metric columns are non-null with a default of zero, so these records trip the zero-emissions rule; most of the 6,105 ERROR flags are of this kind (Section 5.5).
- **Compare dialog label.** The comparison dialog labels CAMPD’s gross load as “Net Generation”. The value is gross load, as everywhere else in the application.
- **Facilities without coordinates.** 37 of the 1,619 facilities have no latitude or longitude and therefore do not appear on the map; they appear in every other view.
- **Dialog accessibility.** The shared dialog component closes on Escape and on an outside click, but does not declare `role="dialog"` or trap keyboard focus, so screen readers do not announce it as a dialog.
- **Language-model fallback.** The failure paths (no key, invalid key, timeout) were tested; the success path depends on an OpenRouter key that was not available during testing.
- **Duplicates tab.** The upload report’s Duplicates tab lists both repeats within the file and rows that already exist in the database (“will update” or “unchanged”). This is intended, but its count differs from the Duplicates tile, which counts only repeats within the file.

6.6 Does the project meet its objectives?

Table 10 assesses each Phase 1 objective from Section 1.

Table 10: Assessment of the Phase 1 objectives.

	Objective	Verdict	Evidence
O1	Web system with front end and database	Met	Sections 2 and 3.
O2	Routes, templates, forms, interactive components	Met	With Next.js and React in place of Flask and Jinja (Section 7).
O3	Relational database with SQLite and an ORM	Met	Six tables, keys, cascades, 14 indexes; Drizzle in place of SQLAlchemy.
O4	Retrieve public data or import files	Met	CAM API retrieval with preview; CSV and Excel upload.
O5	Validate, clean, transform, store	Met	Upload and retrieval validation; issues kept; sanity audits.
O6	Search, filter, sort, paginate	Met	Basic, multi-constraint, ranking, and description search; all results sortable and paginated; Table 8.
O7	Export data and results	Met	Six CSV export types.
O11	Test and deploy	Partly met	Tested as described above. Runs locally from the README; not deployed to a public host.
O12	Document sources, assumptions, equations, units, limitations	Met	README, DATABASE_BREAKDOWN.md, this report.

In summary, every Phase 1 data-management requirement is implemented and tested, and every specification example returns the result an independent SQL query gives. The one objective only partly met is deployment, which was deliberately limited to a local installation.

7 Discussion

7.1 Which techniques and tools worked best for which tasks

SQL for search and ranking. Ranking, the specification’s top- N , bottom- N , and group-and-rank searches, is a natural fit for SQL window functions. Computing ranks in the database, rather than sorting fetched rows in TypeScript, means that pagination, the total count, and the CSV export are all derived from one ranked subquery and cannot disagree. Only one page of rows ever leaves the database. The same reasoning applies to the percentile thresholds of the description search: a single `ORDER BY ... LIMIT 1 OFFSET  $k$` query is both simpler and faster than loading the values.

Python for reading files. The specification requires uploaded files to be read with Python, and the requirement proved sensible. Python’s `csv` module and `openpyxl` handle encodings, byte-order marks, quoted fields, Excel dates, and typed cells with little code. Running the parser as a subprocess with a JSON interface kept the boundary simple. The Python side decides what a valid row is, and the TypeScript side decides what to do with valid rows.

TypeScript for the shared write path. The most valuable structural decision was to give retrieval, upload, and the command-line sync one write path. Diffing against stored records, upserting, deriving rates, auditing, and logging issues are implemented once in `ingest.ts`. When the database-versus-API comparison was added, it worked for uploads at the same time, and a fix to the audit rules applies to every source. Static types made this refactoring safe: changing the shape of a record type surfaced every affected call site at compile time.

Zod at every boundary. Every value that enters the server from outside, whether procedure inputs, CAMPD rows, environment variables, or language-model output, is parsed with a Zod schema before use. Malformed input therefore fails at the boundary with a readable message rather than deep inside a query.

7.2 Important technical choices and alternatives

Next.js, tRPC, and Drizzle instead of Flask, Jinja, and SQLAlchemy. The course suggests a Flask application with Jinja templates and SQLAlchemy; the team’s Phase 1 proposal specified a Next.js stack instead. The motivation was interactivity: the explorer updates results as filters change, keeps state in the URL, and opens nested dialogs, all of which would need a separate JavaScript layer on top of server-rendered Jinja pages. With one language and tRPC, the server’s procedure signatures are the client’s API, so a renamed filter or a changed result field is a compile error rather than a runtime bug. The cost is a second language for file parsing and a subprocess boundary. Each element of the suggested stack still has a direct counterpart: route handlers and procedures for Flask routes, React components for templates, Drizzle for SQLAlchemy, and the same SQLite database. The learning objectives about routes, templates, forms, and an object-relational mapper are therefore met with equivalent tools.

Drizzle instead of Prisma. The proposal named Prisma. Prisma’s query API describes results as nested objects and has no way to express a window function or a correlated subquery except through raw SQL strings, which lose type checking. Drizzle is a thinner layer: queries are composed from typed SQL fragments, so the ranking expression, the per-facility subqueries, and the provenance conditions are ordinary, type-checked code. Drizzle also generates readable SQL migrations from the TypeScript schema.

A rule-based parser instead of a language model. For description search, a language model would handle more varied phrasing with less code. The rule-based parser was chosen as the primary method because it is deterministic, needs no network or key, answers in milliseconds, can be unit-tested exactly, and can only produce values that exist in the database. Its limitation is coverage: phrasing it has no rule for is reported rather than understood. The optional fallback combines the two approaches: the model sees only the leftover words and may only add vocabulary values, so it can extend coverage without making the core behaviour unpredictable.

Preview-then-save instead of direct synchronisation. The first version of retrieval wrote immediately. Replacing it with a preview costs a second request to the EPA API on save, about two seconds for a full year. In return, the user sees exactly what will change before anything is written, and a failure part-way through a fetch can no longer leave a partial dataset. For data whose provenance matters, this is the better default; the command-line sync still writes directly for scripted refreshes.

One page with dialogs instead of separate pages. The specification lists six pages. Implementing them as views and dialogs within one page keeps the user’s search in place while they retrieve, upload, inspect, or download, and every search remains reproducible through its URL. The trade-off is that the dialogs themselves are not addressable by URL.

Annual data stored, sub-annual data fetched. Storing hourly data for every unit would mean tens of millions of rows for one year. Since Phase 1 searches operate on annual records, only those are stored; the granular views fetch hourly to monthly data from the API on demand and cache it in the browser.

7.3 Challenges and lessons learned

Combined attribute values. CAMPD’s master-data endpoints list single values, such as “Coal” or “Wet Limestone”, but unit records often contain combinations, such as “Coal, Pipeline Natural Gas” or “Wet Lime FGD|Wet Limestone”. An equality filter therefore missed many units. The solution was to split combined values into distinct options for the dropdowns and to match fuel, type, and control filters by case-insensitive substring. The same issue means the “Already in the database” panel can only approximate what an API request with a fuel or control filter will return; the dialog says so.

Superseded datasets. Because records are keyed by facility, unit, and year, re-retrieving a year moves its records to the new dataset and leaves the old one owning nothing. Deleting the old dataset would erase history, and counting it as active would double the coverage figures. The adopted rule labels an API dataset *superseded* when it once stored records but now owns none. Only active datasets are counted, and superseded ones remain visible in the retrieval history.

An ORM rendering surprise. The first implementation of that rule returned wrong results. Drizzle omits table prefixes from column names in single-table queries, so in `SELECT ... FROM datasets WHERE NOT EXISTS (SELECT 1 FROM annual_records WHERE dataset_id = id)` the unqualified `id` bound to `annual_records.id` instead of `datasets.id`. The fix was to qualify these subqueries by hand, with a comment explaining why. The lesson was to read the SQL an ORM generates whenever a query relates two tables in a non-trivial way.

Ambiguity in natural language. Several two-letter state codes are also common English words: “in”, “or”, “me”, “ok”. Treating them as states would turn “units in Texas” into a search for both Indiana and Texas. The parser now accepts these codes only in capitals, while full state names and unambiguous codes work in any case. A similar ambiguity existed in the data: the status “Operating (Retired 04/15/2016)” contains the word “Retired”, so the status filter uses a prefix match instead of a substring match.

How far EPA’s data extends. CAMPD publishes the current year quarterly, and requests beyond the last published quarter fail. The API offers no endpoint that states this date, but its error message names it (“... the quarter ending on 06/30/2026”). The client therefore probes with a full-year request, parses the date from the error, caches it for an hour, and uses it to limit the date pickers of the granular view. If EPA publishes a new quarter while the application runs, a failing request reveals the new date and is retried once.

Performance needs measurement. The year-filtered ranking query described in Section 6 ran for nine seconds before anyone looked at its query plan, because every query was fast without a year filter. One composite index fixed it. The lesson is that an index on each foreign key is not enough; indexes should match the combinations of columns that queries actually filter on together, and the planner’s choice should be checked rather than assumed.

General lessons. Three practices paid off repeatedly. Keeping logic pure, with no database or network, made it testable with fast unit tests. Having one code path for each concern, such as one write path, one CSV writer, and one ranked query per view, meant that fixes applied everywhere at once. And refusing to discard data, by keeping rejected rows, flagged records, and superseded datasets, made every count in the interface explainable.

8 Limitations and future improvements

8.1 Current limitations

Unit attributes are not versioned by year. Fuel, unit type, controls, and program codes are stored once per unit, and the latest import’s values win. The specification places the control and program information on the annual record. In practice CAMPD reports them per unit and they change rarely, but when they do, history is lost. Big Sandy unit BSU1, for example, reported coal as its primary fuel for 2014 but natural gas today. `epaData` would show it as a gas unit in every year, so a search for coal units in 2014 would miss it.

Records are overwritten, not versioned. Each facility-unit-year has one stored value. Re-importing it overwrites the metrics and moves the record to the new dataset. The dataset’s counts record *that* a change happened, and the preview shows *what* changed before saving. After saving, however, the previous value is gone.

Origin is known per unit-year only. Facility and unit rows have no link to the dataset that created or last modified them, so the Origin filter applies to annual records only.

“Not reported” is stored as zero. Missing metric values are stored as zero. For heat input or gross load this is usually correct, but a missing CO₂ mass becomes a reported zero, which the zero-emissions rule then flags (Section 5.5).

Local, single-user deployment. The application runs on one machine against one SQLite file and has no user accounts: anyone who can open it can upload files and run retrievals. An early attempt to host it on a serverless platform required copying the database to temporary storage on every cold start, which loses writes, and was abandoned.

Description search has fixed coverage. The parser does not express negation (“not retired”), alternatives (“coal or gas”), or comparisons between entities (“units larger than Gavin”); such phrases are reported as not understood. Facility names are only recognised after cue words such as “owned by” or “named”.

Upload formats. Uploads recognise the CAMPD annual-emissions layout and the facility-attribute layout. Hourly and daily files are read, but only their facility and unit attributes are stored, because sub-annual metrics have no place in the schema.

Planned features not built. Map clustering and year-over-year spike detection from the proposal were not built (Table 2).

8.2 Future improvements

Phase 2: evaluation. The next phase adds the remaining tables of the specification: TRACI characterisation factors, indicator definitions, calculated indicators, weight scenarios with their weights, and score results. The existing design supports this directly. Indicators are functions of the stored annual records, for example SO₂ mass multiplied by its acidification factor. They can be computed with the same ranked queries and exported through the same CSV path, and each indicator row can reference the dataset of the records it was computed from.

Data model.

- A `unit_year_attributes` table holding fuel, controls, and programs per unit and year, filled from the annual API rows that already contain them.
- A history table to which every overwritten annual record is copied together with the dataset that replaced it, making changes auditable after saving.
- Nullable metric columns, or a “reported” flag, to distinguish a missing value from a reported zero, and a corresponding refinement of the zero-emissions rule.

Data quality. Year-over-year anomaly detection, flagging a unit whose emissions change by more than a set number of standard deviations of its own history, would complement the physical rules. A review state on audit flags (open, accepted, or explained) would let analysts record their conclusions.

Search. Negation and alternatives can be added to the filter model as “exclude” and multi-value filters, after which the parser can map “not” and “or” onto them. Saved searches with names would complement the shareable URLs.

Deployment and access. Hosting the database on a libSQL server, which the client library already supports, would allow a shared deployment. That would also require user accounts, with upload and retrieval restricted to authorised users, and background jobs for multi-year retrievals. The dialogs should gain proper dialog semantics and focus management for assistive technologies.

9 Individual contributions

The Phase 1 proposal divided the work into a back-end role and a front-end role, with integration, testing, sample data, and documentation shared. The paragraphs below state each member’s responsibilities and the completed work as recorded in the project’s version history.

Raaj Banga (back-end lead). Responsible for the database schema and indexing, the CAMPD API integration, data validation and anomaly detection, and the search and pagination API. Completed work, by the repository history:

- the schema, its six migrations, and the index design, including the (`facility_id`, `year`) index added after profiling;
- the CAMPD client: annual retrieval, the preview and comparison with the database, validation and issue logging for API rows, master-data options, the granular time series, and detection of EPA’s published-through quarter;
- the shared write path (diff, upsert, derived rates, physical-sanity audits) and the provenance model (dataset counts, superseded datasets, record origin);
- the search back end: filter, range, and ranking queries, URL state, description search with its percentile thresholds and optional language-model fallback, and the CSV export service;
- the explorer interface, map and globe, detail, comparison, retrieval, and download dialogs, and the shared UI components;
- the unit tests, the sample upload files, the README, `DATABASE_BREAKDOWN.md`, the demo script, and this report.

Arnav Bawankule (front-end lead). Responsible for the user-interface layout, filter forms, map and comparison views, upload dialogs, and export controls. Completed work, by the repository history:

- the CSV and Excel import feature: the Python parser that maps file columns to the schema and validates every row, the data-quality report of rejected and duplicate rows, the upload endpoint, and the upload dialog with its preview and approval step, together with its first tests.

The presentation and live demonstration are shared: Arnav presents the interface, upload, detail, and comparison steps, and Raaj the retrieval, search, ranking, and download steps.

10 References and acknowledgments

10.1 Data

All emissions, operating, facility, and unit data come from the U.S. Environmental Protection Agency’s Clean Air Markets Program Data, retrieved through the CAM API (<https://www.epa.gov/power-sector/cam-api-portal>) between September and October 2026. The data is a U.S. government work in the public domain. The rows of the sample upload file are CAMPD data for Kentucky (reporting years 2014 and 2023), retrieved on 9 October 2026, with five rows deliberately altered for testing as documented in `samples/README.md`.

10.2 Reused code and assets

- The project was scaffolded with create-t3-app (<https://create.t3.gg>), which provided the initial configuration of Next.js, tRPC, Drizzle, Tailwind CSS, and the environment-variable validation. None of its example code remains apart from that configuration.
- U.S. state and world outlines for the globe come from the us-atlas and world-atlas TopoJSON packages by Mike Bostock (ISC licence; <https://github.com/topojson/us-atlas>), bundled in `public/geo/`.
- Map tiles are provided by CARTO and OpenStreetMap contributors.
- All other code depends on the open-source libraries listed in Section 4 and `package.json`, used under their respective licences.

10.3 Generative AI disclosure

Generative AI was used throughout this project, in line with the course policy on its disclosure. We used Anthropic’s Claude, through the Claude Code tool, to help write, refactor, and review application code and tests; to investigate defects; to draft and revise the README, the database documentation, the demo script, and this report; and to capture the screenshots. The proposal stated its own use of AI in the same way.

The team defined the requirements and design decisions, reviewed the generated code and text, and accepted changes only after checking them. The checks were: the type checker, linter, and unit tests; the acceptance queries against hand-written SQL (Section 6); and manual use of the application. Every number in this report was produced by running the application or a SQL query against the committed database. The team takes responsibility for the content of the code and of this report.

Bibliography

- [1] Jane Bare. TRACI 2.0: The tool for the reduction and assessment of chemical and other environmental impacts 2.0. *Clean Technologies and Environmental Policy*, 13(5):687–696, 2011.
- [2] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. D³: Data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011.
- [3] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001.

- [4] Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [5] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.
- [6] Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, 15(12):3535–3547, 2022.
- [7] Rob J. Hyndman and Yanan Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996.
- [8] Viktor Leis, Kan Kundhikanjana, Alfons Kemper, and Thomas Neumann. Efficient processing of window functions in analytical SQL queries. *Proceedings of the VLDB Endowment*, 8(10):1058–1069, 2015.

We used AI to help with this project.